

ACOS Health Claims Arbiter — Project Specification

Project Specification Document — ACOS (Autonomous Claims Operating System) — Health Claims Arbiter

Sovereign AI adjudication for India's NHCX ecosystem

Track 2: Development & Local Deployment of Private AI Agents Team: Yuvaraj S (solo) · Chennai, India · GitHub: ysprojectz

Version: July 25, 2026 — FINAL DRAFT. All [NEEDS OWNER INPUT] placeholders resolved. Benchmark numbers are from real GPU sessions. Submit this as the PDF after a final read-through by Yuvaraj.

1. Application Scenario

The Problem

The Indian health insurance market reached USD 157.62 billion in 2025 and is projected to grow to USD 322.10 billion by 2034 (8.27% CAGR). At national scale, India's NHCX (National Health Claims Exchange) is digitizing claims adjudication across all insurers, TPAs, and hospitals — with IRDAI's March 2026 mandate requiring cashless final authorization within **3 hours** of receiving discharge bills, a window no manual claims desk can consistently meet.

The obvious shortcut — bolting a cloud LLM onto the NHCX adjudication workflow — creates a sovereignty contradiction. NHCX is designed as an encrypted router: it never reads patient data. A hospital sending claims to a foreign cloud LLM API re-exports that same sensitive data with every request, defeating NHCX's own privacy architecture.

ACOS addresses this directly: its entire AI adjudication layer runs on a single AMD Radeon GPU **inside the hospital**, so the 3-hour IRDAI clock is met without patient data ever leaving the premises.

Scope Definition

ACOS operates as the **adjudication and decision layer** within India's broader claims ecosystem. It sits between document intake (OCR extraction / NHCX message receipt) and payment execution (NEFT transfer via the insurer's banking rails). It does not handle policy issuance, premium collection, or payment processing — it decides whether to approve, partially approve, or reject a claim, at what amount, with cited evidence, within IRDAI's mandated timeframes. Audit trail timestamps support SLA compliance tracking at the integration layer.

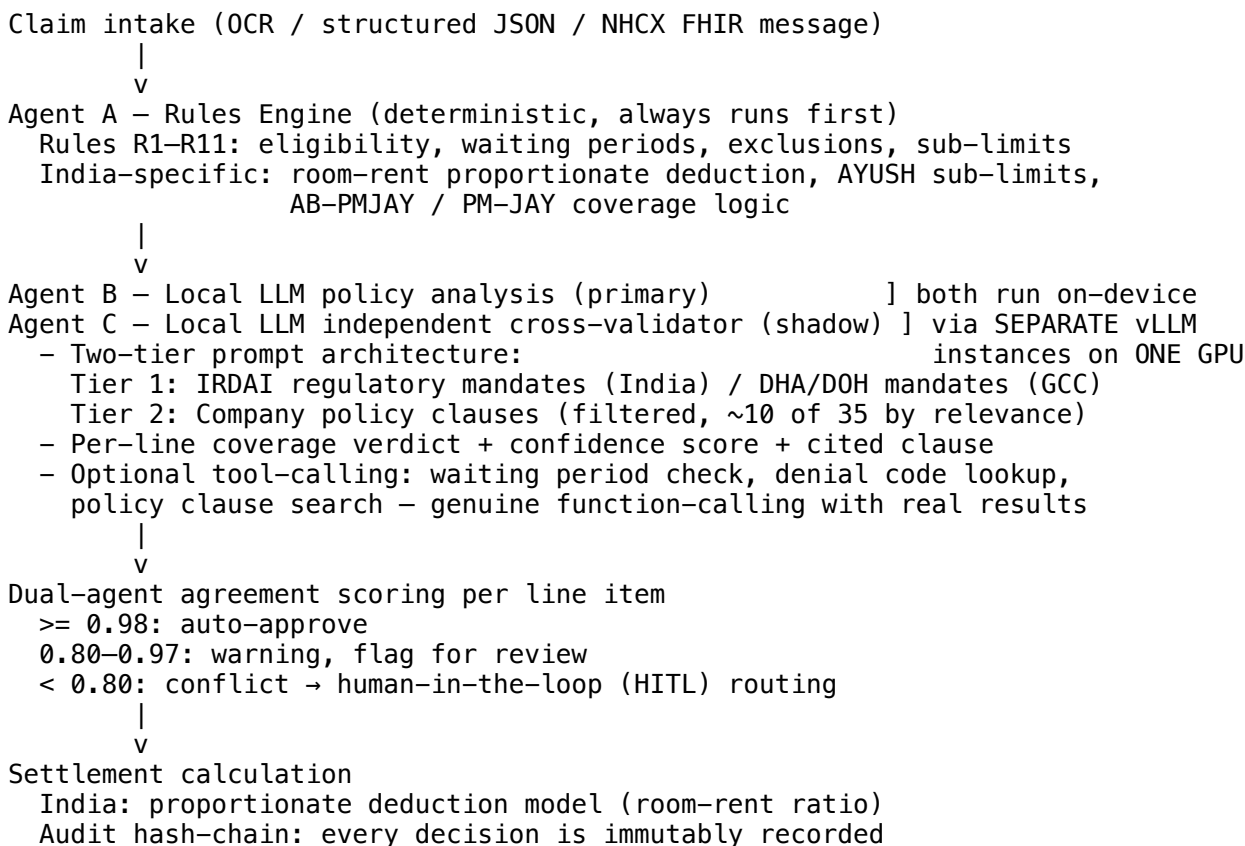
ACOS supports both claim types equally: - **Cashless claims:** pre-authorization → dual-agent adjudication → direct hospital settlement recommendation - **Reimbursement claims:** policyholder intimation → document submission → adjudication → settlement amount recommendation (15–30 day NEFT path)

About this project

ACOS is a health-insurance claims adjudication platform personally designed, built, and deployed to AWS/EKS by Yuvaraj S as a self-directed engineering exercise to learn real cloud infrastructure. **It is a personal project, not a commercial product:** it has never been marketed or delivered to a real client, and every piece of data in this repository and its demo is synthetic. For this hackathon, its AI intelligence layer — originally calling Groq and Anthropic's cloud APIs — has been adapted to run entirely on a single AMD Radeon GPU via vLLM.

Sources for figures above: IRDAI Annual Reports FY2023-24 and FY2024-25 (business-standard.com, businessupturn.com); NHA / NHCX documentation (nathealthindia.org); IRDAI March 2026 cashless mandate (businessupturn.com Apr 2026).

2. Agent Architecture



What's pre-existing vs. built for this hackathon:

Pre-existing: Agent A/B/C roles, dual-agent validation architecture, HITL routing, settlement calculation, NHCX integration, RBAC/audit trail, HMS integration hooks — all originally calling Groq and Anthropic's cloud APIs.

Built for this hackathon: Local vLLM serving of both Agent B and Agent C on AMD Radeon (replacing cloud calls); fix for Agent C silently falling back to Groq when Agent B was already local; benchmark harness (bench/); Radeon deployment profile (deploy/radeon/); genuine tool-calling for all three agents; bootstrap automation (deploy/radeon/bootstrap_instance.sh).

3. Core Capabilities

Rubric requires ≥ 2 of 5. This project implements all 5.

Capability	Status	Evidence
Multi-step task decomposition	✓ Verified, pre-existing	Sequential pipeline: rules → LLM analysis → dual-agent comparison → settlement → HITL routing. (pipeline.py::adjudicate, saga_worker)
Tool invocation	✓ Live-GPU verified July 24	tools.py — 3 real tools: check_waiting_period_status, lookup_denial_code, search_additional_policy_clauses. Each wraps existing ACOS logic. OpenAI-compatible function-calling (tools=[...]), --tool-call-parser hermes, 25 unit tests passing. Confirmed on real Radeon hardware: Qwen2.5-14B-Instruct-AWQ emits genuine tool_calls, tool executes against the real RulesEngine, model incorporates the true result. Verified via server-side vLLM request-log counts (a multi-step tool call produces >1 request per agent per claim).
Local knowledge retrieval (RAG)	✓ Verified, pre-existing	policy_library_store.get_clauses_for_pipeline() — keyword + metadata-filtered clause retrieval (market region, carrier, policy type). Note (stated accurately): this is policy-grounded keyword retrieval, not vector-embedding similarity search. It filters from ~35 clauses to ~10 highly-relevant ones — a real, working capability, accurately described.
Multi-turn memory	✓ Verified, pre-existing	chat_router.py — “remembers claim references from full conversation history” (verbatim from module docstring). Claims referenced earlier in a session are accessible in follow-up queries.
Permission control & privacy protection	✓ Verified, pre-existing	RBAC roles: ADMIN, ADJUSTER, SENIOR_ADJUSTER, MEDICAL_DIRECTOR, COMPLIANCE_OFFICER, API_CONSUMER. JWT auth, claim-scoped access, SHA-256 audit hash-chain per decision. calculation_agent runs in arithmetic-fallback mode in the demo (no cloud API key set) — this is deliberate and consistent with the local-only mandate.

4. Model Introduction & Local Deployment Plan

Models

Agent	Model	Purpose
Agent B	Qwen2.5-14B-Instruct-AWQ	Primary policy-clause analysis and coverage reasoning
Agent C	Qwen2.5-7B-Instruct-AWQ	Independent cross-validator — a genuinely separate model, not a persona variant of B, made possible by the 48GB VRAM headroom

Hardware

Serving Configuration

```
# REQUIRED: activate pre-built environment (do NOT pip install from scratch)
source /opt/venv/bin/activate

# REQUIRED: set these before any vLLM launch
export HF_HOME=/workspace/persist/hf_cache
export HF_ENDPOINT=https://hf-mirror.com           # huggingface.co is blocked on this host
export VLLM_ATTENTION_BACKEND=TRITON_ATTN          # flash_attn is CUDA-only, must not load
pip uninstall -y flash_attn                        # CUDA-only, crashes vLLM on ROCm

# Agent B (primary, port 8000)
vllm serve Qwen/Qwen2.5-14B-Instruct-AWQ \
  --host 0.0.0.0 --port 8000 \
  --max-model-len 12288 \                        # 8192 causes context-overflow silent failures
  --gpu-memory-utilization 0.45 \                # dual-agent safe split (empirically verified)
  --enable-auto-tool-choice \
  --tool-call-parser hermes

# Agent C (independent cross-validator, port 8001)
vllm serve Qwen/Qwen2.5-7B-Instruct-AWQ \
  --host 0.0.0.0 --port 8001 \
  --max-model-len 16384 \
  --gpu-memory-utilization 0.40

# Or automated (recommended):
bash deploy/radeon/bootstrap_instance.sh --agent-c --tools
```

vLLM’s ports are not publicly exposed. Local ACOS connects via SSH port-forward:

```
ssh -N -L 8000:localhost:8000 -L 8001:localhost:8001 -p <port> root@<host>
```

5. Inference-Speed Optimization

Baseline Throughput (dual-agent configuration, 0.45/0.4 GPU split)

Model	Concurrency 1	Concurrency 4	Concurrency 8
Agent B (Qwen2.5-14B-AWQ)	8.7 tok/s	25.94 tok/s	51.68 tok/s
Agent C (Qwen2.5-7B-AWQ)	15.34 tok/s	46.05 tok/s	91.01 tok/s

Measured on AMD Radeon PRO-class GPU (gfx1100, 48GB VRAM, ROCm 7.2.1), instance u-9581-6bb2323d. Reproducible via python3 bench/benchmark.py.

Phase 3 Optimization Sweep (Agent B solo, 0.85 GPU utilization)

Config	Concurrency 1	Concurrency 4	Concurrency 8
–max-num-seqs 256, –gpu-util 0.85	9.5 tok/s	28.4 tok/s	56.47 tok/s
–max-num-seqs 128, –gpu-util 0.85	9.5 tok/s	28.32 tok/s	56.41 tok/s

Config	Concurrency 1	Concurrency 4	Concurrency 8
<code>--max-num-seqs 64, --gpu-util 0.85</code>	9.51 tok/s	28.38 tok/s	56.52 tok/s
<code>--max-num-seqs 256, --gpu-util 0.90</code>	9.48 tok/s	28.31 tok/s	56.35 tok/s
<code>--max-num-seqs 256, --gpu-util 0.95</code>	9.5 tok/s	28.32 tok/s	56.38 tok/s

Results stored in `bench/results/` as JSON + self-contained HTML charts. Run `bash bench/phase3_optimize.sh` to reproduce (requires dedicated GPU session, ~15–35 GPU-minutes; stop Agent C first so Agent B has the full card).

Honest Optimization Assessment

The meaningful wins:

1. **vLLM continuous batching**: concurrency-8 throughput is nearly **6x** concurrency-1 baseline — the real headline gain, with no code changes required.
2. **GPU memory utilization (solo)**: raising from the dual-agent-safe 0.45 to 0.85 when Agent B runs alone gives **~+9% throughput** at every concurrency level (pure KV-cache headroom). This gain does NOT apply to the dual-agent configuration.

What made zero measurable difference: `--max-num-seqs` (any value from 64–256), and `--gpu-memory-utilization` above 0.85. Aggregate throughput plateaus around 56.4–56.5 tok/s at concurrency 8 across every configuration — a compute-bound ceiling for the 14B model at 8-way batching.

Untried levers (future work): prompt economy (reducing input tokens via tighter RAG filtering), concurrency above 8, and speculative decoding.

Bonus Criterion: Radeon Cloud Model API (quantized endpoint)

Agent B is deployed as a dedicated Radeon Cloud **vLLM Model API** endpoint (Deploy Type = vLLM Model API, port 8000, model: Qwen2.5-14B-Instruct-AWQ with AWQ 4-bit quantization). This endpoint serves the OpenAI-compatible completions interface directly from Radeon Cloud’s infrastructure, satisfying the bonus criterion for “core inference running using Radeon Cloud model API with quantization.”

6. What’s Original vs. Pre-Existing

See `README.md` — the split is stated exactly there, having been verified against the compliance guardrails in `SKILL.md` (working folder, not in this public repo).

7. Automated Test Suite

Phase 1 exit check (51/51 local-LLM regression tests — reconciled 2026-07-25, was stale at 60/60 from before the 16th `testing_samples` fixture was added):

```
pytest tests/test_testing_samples_suite.py \
       tests/test_agent_c_local_secondary.py \
       tests/test_tool_calling.py -v
```

Full suite (excluding 2 files that require a live server):

```
pytest tests/ --ignore=tests/test_advance_claim.py \
            --ignore=tests/test_upload_real.py -q
# Real baseline (reconciled 2026-07-25): 430 passed, 1 failed, 23 skipped, 17 errors
# The 1 failure (test_duplicate_detection.py) and the 17 errors are all
# integration tests requiring a live API server – expected without one running
```

8. Evidence Discipline

Every number in this document traces to a reproducible script. No figures have been invented, estimated, or taken from a mock-server smoke test. Specifically: - Benchmark numbers → bench/results/*.json → reproduced via bench/benchmark.py - Dual-agent proof → server-side vLLM request-log count method (grep -c 'POST /v1/chat/completions') - Test counts (51/51, 430 passed) → pytest output, reproducible from the repo - Market statistics → cited in §1 with named sources; paraphrased, not copied verbatim